P2 Poster Presentation: Speech and Guide

Design and Evaluation of a Drift-Aware Continuous Learning MLOps Framework for Financial Transaction Fraud Detection

Mashud Hasan (22299053)
Nirnoy Biswas (24241185)
Md. Sabbir Hossain (22299038)
Al Imran (231011035)
Imtiaz Zaman Sami (23101551)

Supervisor: Md. Sabbir Ahmed, Senior Lecturer
Co-supervisor: Naima Tahsin Nodi, Adjunct Lecturer

Department of Computer Science and Engineering, Brac University

Part A. How to use this file

The speech in Part B is about 925 words, which is about 6 minutes at a steady pace. Part C explains every block and every figure of the poster in simple words, so anyone on the team can answer questions about any part. Parts D to F are a cheat-sheet: numbers, simple meanings of terms, and likely questions.

Time	Section	Poster block to point at	Suggested speaker
0:00 - 0:45	Opening and problem	Title, Abstract, Problem Statement	Speaker 1
0:45 - 1:35	Objectives and literature	Research Objectives, Literature Review	Speaker 1
1:35 - 2:25	Datasets and preprocessing	Data set table, preprocessing diagram	Speaker 2
2:25 - 3:40	Method, model, framework	Three Methodology diagrams	Speaker 3
3:40 - 5:15	Results	Grey Result Analysis boxes	Speaker 4
5:15 - 6:00	Conclusion and future work	Conclusion & Future Work	Speaker 5

Speakers are only a suggestion; one person can give the whole speech.

Tips: stand beside the poster, not in front of it. Point at each figure before you talk about it. Say the one-line takeaway of a graph first, then one number. If you forget a number, the takeaway is enough. If a question is about a limitation, admit it and say how the future work handles it.

Part B. The speech

Read at a calm speed (about 150 words a minute). Orange lines are stage cues: do not read them aloud. Bold words are the ones to stress.

0:00 - 0:45 | Opening and the problem

[Stand beside the title. Smile, look at the audience, not the poster.]

Good morning. We are Mashud, Nirnoy, Sabbir, Imran and Sami. Our thesis is a **drift-aware continuous learning MLOps framework for financial fraud detection**, supervised by Md. Sabbir Ahmed and co-supervised by Naima Tahsin Nodi.

Banks use machine learning models to catch fraudulent transactions. Normally the model is trained once and tested once. But in real life two things go wrong. First, fraudsters keep changing their tricks, so the data slowly changes and a good model becomes weaker. This is called **concept drift**. Second, the bank only finds out that a transaction was fraud weeks later, when the customer complains. This is called **label delay**. So the model cannot see its own mistakes in time.

[Point at the pink Problem Statement box.]

So our question is simple: **when should a fraud model be retrained, and how do we stop a bad new model from going live?**

0:45 - 1:35 | Objectives and literature review

[Point at Research Objectives, then Literature Review.]

We had five objectives: measure how a deployed model gets worse and whether drift alarms notice it; measure how label delay reduces the benefit of retraining; compare scheduled and drift-triggered retraining fairly; build a full MLOps framework and test it with injected faults; and repeat everything on three datasets.

From the literature: Dal Pozzolo and colleagues showed that fraud labels arrive late. Menezes and Filho showed that fraud models lose up to 40 percent of their F1-score under natural drift, but they did not test retraining. Shakil and colleagues built drift-triggered retraining for MLOps, but only on artificial drift with instant labels. So the **gap** is that nobody had fairly tested retraining policies for fraud with real drift and late labels.

1:35 - 2:25 | Datasets and preprocessing

[Point at the dataset table, then the preprocessing diagram.]

We used three public datasets from Kaggle. **IEEE-CIS** is real online payments: about 590 thousand transactions over six months, 3.5 percent fraud. **Sparkov** is simulated card payments: 1.8 million transactions over two years, only half a percent fraud. **BAF** is bank account applications: one million rows over eight months, about one percent fraud. Using three different datasets shows our results are not luck on one dataset.

For preprocessing, we merged and cleaned the files, removed personal information, and built features like hour, age, distance and card history, using only **earlier** transactions so the model never sees the future. Then we sorted everything by time and never shuffled. The first half is used to train the first model, and the rest is replayed week by week, just like real life.

2:25 - 3:40 | Methodology, model and framework

[Point at the workflow diagram (8 boxes).]

Our study had eight steps: literature review, data preparation, a static baseline, a retraining simulation with label delays from 0 to 60 days, statistical checks with confidence intervals and five random seeds, then building the framework, testing it with faults, and replicating on all datasets.

[Point at the model architecture diagram.]

Our model is **LightGBM**. It builds many small decision trees one after another, and each new tree fixes the mistakes of the earlier ones. Their outputs are added and turned into a fraud probability between 0 and 1. We chose it because it is fast, strong on table data, and handles missing values. We trained it ourselves; no pre-trained model is used.

[Point at the framework diagram and follow the arrows with your hand.]

Our framework runs as a weekly loop. Transactions are scored by an API. Labels arrive late into a label store. The **monitor** checks performance on the labels that have arrived. The **retrain policy** retrains every 14 days, with a safety net if performance suddenly crashes. The trainer tries two training windows, the last 60 days and all history, and keeps the better one. Then the **promotion gate** compares the new model with the current one on unseen data, and only lets it go live if it is not worse. MLflow keeps every version, so we can roll back, and a dashboard shows every decision.

3:40 - 5:15 | Results

[Point at the first result figure (top: lines, bottom: bars).]

Our main score is **PR-AUC**: how well the model puts the real frauds at the top of the list. First result: drift alarms do not track performance. On IEEE-CIS, performance falls from 0.58 to 0.39, but the drift score never crosses the alarm line. That is a **missed alarm**. On BAF, the drift score is huge, but performance stays flat. That is a **false alarm**.

[Point at the label-delay figure.]

Second: label delay eats the benefit of retraining. On IEEE-CIS, retraining improves PR-AUC by 0.078 when labels arrive at once, but only by 0.010 when they arrive 60 days late. The window also matters: on Sparkov, a short window actually hurts.

[Point at the window-selection bars.]

Third: our automatic window choice matched the best fixed window on every dataset, without knowing in advance which one was right.

[Point at the gate figure.]

Fourth: we deliberately broke the data, for example by shuffling labels. Without the gate, performance dropped by up to 0.080. With the gate, only 0.012, because the bad models were blocked.

And as our supervisor asked, we compared the two policies with the **same number of retrains**. With delayed labels, the schedule still matched or beat the drift trigger, so it does not win just because it retrains more often.

5:15 - 6:00 | Conclusion and future work

[Point at Conclusion and Future Work.]

To conclude: when labels are late, drift alarms can miss real damage, and a simple schedule with the right training window is more reliable. In our work, **drift-aware** means watching drift through delayed-label performance, not through label-free alarms. Window selection and the promotion gate make retraining adaptive and safe, and the results held on three datasets.

Our supervisor also asked us to load test the scoring API. The first test found a bottleneck; after fixing it, the API handles about **1,300 requests per second** and scores one transaction in about a millisecond, with exactly the same scores.

Next, we will finish the equal-budget test on Sparkov. Later we want to try faster investigator feedback, other models and a live deployment. Thank you. We are happy to take your questions.

Part C. The poster explained, block by block

Each block of the poster in simple words. For every figure: what it shows, how to read it, the numbers to remember, and one line you can say about it.

1. Abstract and Problem Statement

What the work is: a system that keeps a fraud model healthy after it is deployed. It watches the model, retrains it on a schedule, picks how much past data to learn from, and only lets a new model go live if it passes a check (the promotion gate).

Why it is needed: most papers (1) test on a random split, which mixes past and future and hides drift, (2) assume labels arrive instantly, which never happens with fraud, and (3) trigger retraining with drift detectors tested on fake, injected drift. We test under realistic conditions instead.

2. Research Objectives

#	Objective	Where the answer is on the poster
1	How a deployed model decays, and whether drift measures detect it	Result figure 1 (decay)
2	How label delay limits the benefit of retraining	Result figure 2 (delay)
3	Compare scheduled and drift-triggered retraining fairly	Conclusion (equal-budget point)
4	Build the framework and test it with injected faults	Framework diagram, result figures 3 and 4
5	Replicate on three datasets	Every result figure has three panels

3. Literature Review (three papers on the poster)

Paper	What they found	What was missing (our gap)
Dal Pozzolo et al. (2018)	Fraud labels arrive late (verification latency)	Did not test MLOps retraining policies
Menezes and Filho (2025)	Fraud models on IEEE-CIS lose up to 40% F1 under natural drift	Measured the damage, did not test retraining
Shakil et al. (2025)	Drift-triggered retraining inside an MLOps pipeline	Only artificial drift and instant labels

Gap in one line: no fair test of retraining policies for fraud under real drift and label delay.

4. Data set table

	IEEE-CIS	Sparkov	BAF
What it is	Real e-commerce payments (Vesta, Kaggle competition)	Simulated credit-card payments (Sparkov generator)	Bank account opening applications (NeurIPS 2022)
Rows	590,540	1,801,969	1,000,000
Time span	182 days	720 days	8 months

	IEEE-CIS	Sparkov	BAF
Fraud rate	3.50%	0.53%	1.10%
Features	431	16	29
Why we chose it	Real drift; the model really decays	Long time span with real timestamps (supervisor asked for a second timestamped dataset)	Built to contain monthly drift; a different kind of fraud

How to read it: each column is one dataset. The fraud rate shows how rare fraud is: in Sparkov only about 5 in every 1,000 transactions. 'Features' is the number of input columns the model uses. All three are table data with a yes/no fraud label.

5. Preprocessing diagram

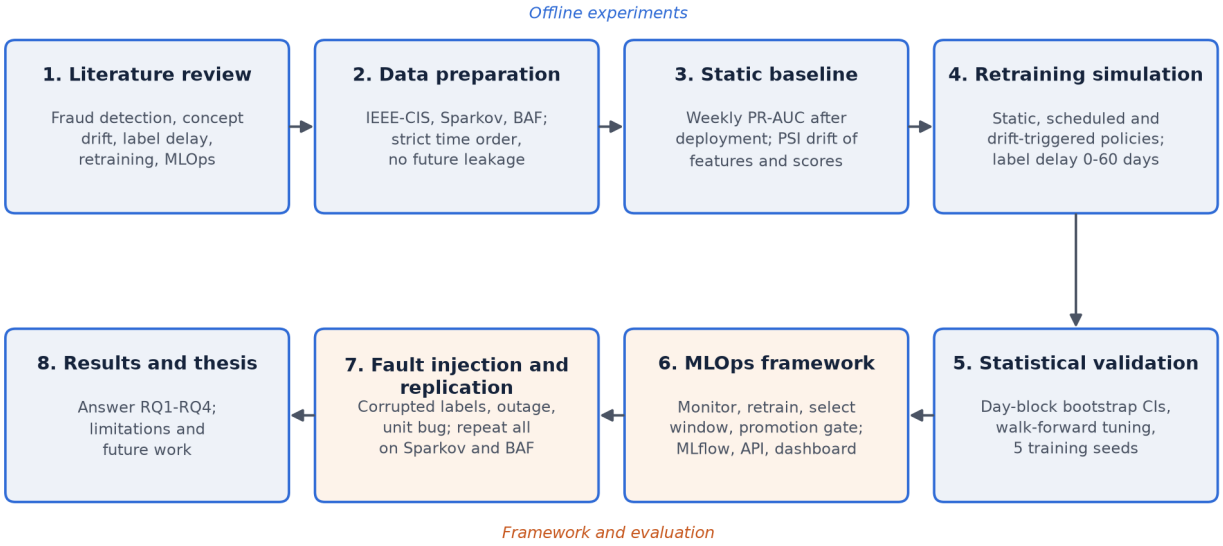


output: one table per dataset with TransactionID, TransactionDT (seconds), isFraud and the model features

What it shows	The five steps from raw downloaded files to the clean table that every experiment uses.
How to read	Left to right. (1) Raw files. (2) Merge and clean: join the two IEEE tables, remove names and card numbers, treat -1 in BAF as missing, cut Sparkov at 21 Dec 2020 because fraud stops there. (3) Make features (hour, weekday, age, distance, card history) from earlier transactions only. (4) Sort by time, never shuffle. (5) Split: the first 50-60% trains the first model, the rest is replayed week by week.
Why it matters	Shuffling would let the model see the future and would hide drift. Using only earlier transactions for features avoids data leakage.

Say it in one line: We kept strict time order so the experiment behaves like a real bank.

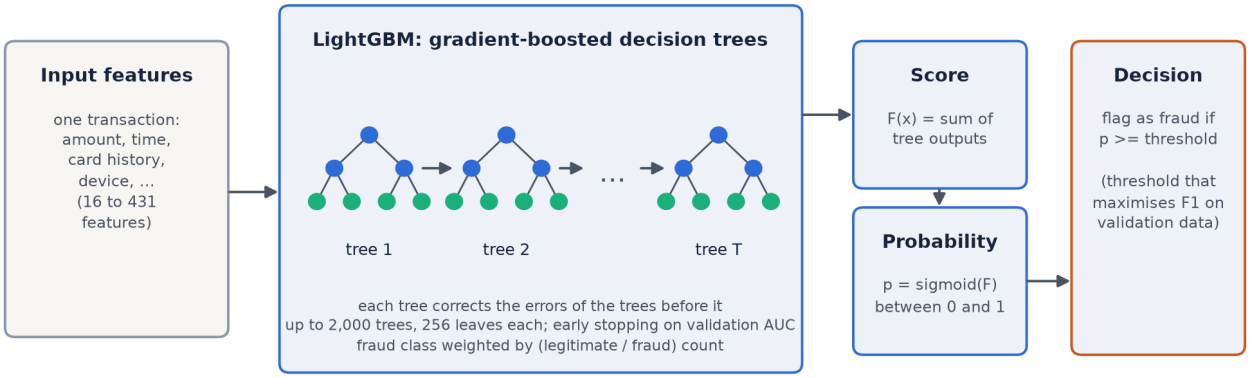
6. Workflow of the study



What it shows	The eight steps of the whole project.
How to read	Top row left to right (offline experiments): literature, data, static baseline, retraining simulation. Then down and right to left (framework and evaluation): statistical validation, MLOps framework, fault injection and replication, results.
Key terms	**Static baseline**: train once, never update, and watch it decay. **Bootstrap CI**: resample the days many times to see how sure a difference is. **Walk-forward tuning**: choose settings using only the past, then test on the next period. **5 seeds**: repeat training with 5 random starts to check the result is not luck.

> **Say it in one line:** First we measured the problem, then built the framework, then tried to break it.

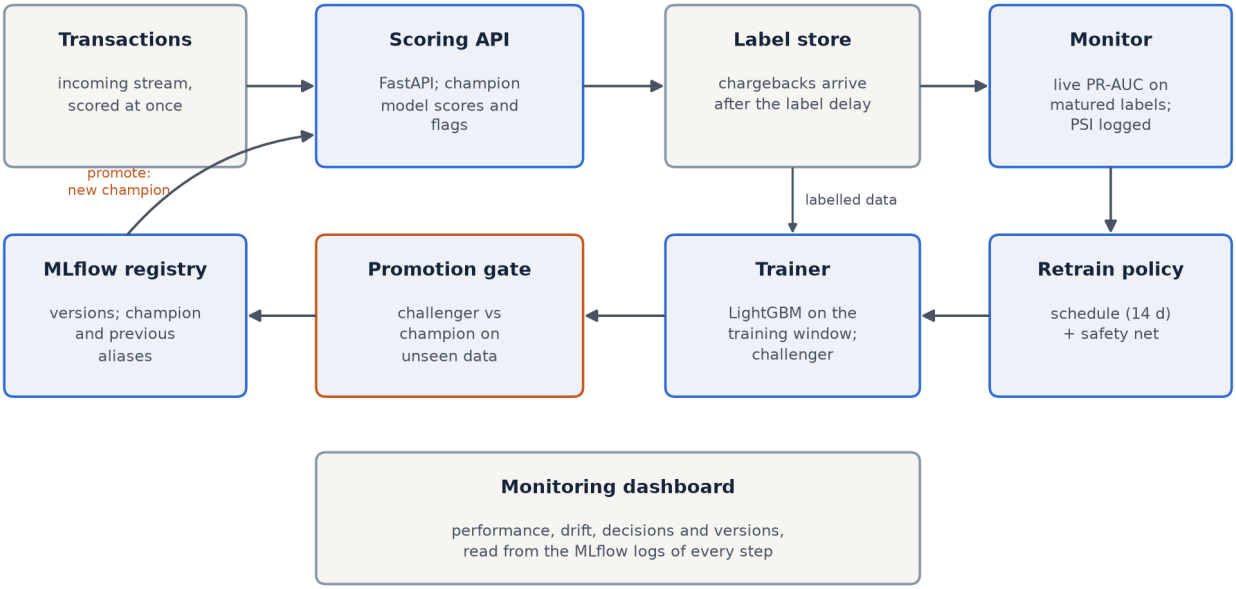
7. Model architecture (LightGBM)



What it shows	How one transaction becomes a fraud decision.
How to read	Input features (16 to 431 columns) go into many small decision trees. Tree 2 learns from the mistakes of tree 1, tree 3 from tree 2, and so on (this is 'gradient boosting'). The outputs are added into a score, the score is turned into a probability between 0 and 1, and the transaction is flagged if the probability is above a threshold.
Settings	Up to 2,000 trees, 256 leaves each, learning rate 0.05, stops early when validation stops improving. The rare fraud class gets a bigger weight instead of creating fake fraud rows (no SMOTE).
Why LightGBM	Fast enough to retrain hundreds of times, best-known performer on table data, handles missing values and categories by itself. Trained from scratch, not pre-trained.

Say it in one line: Many small trees, each fixing the last one's mistakes, give a fraud probability.

8. Proposed framework



What it shows	Our MLOps system and how data flows through it every week.
How to read	Follow the arrows. **Transactions** are scored by the **Scoring API** (FastAPI). True labels arrive late in the **Label store**. The **Monitor** measures live PR-AUC on labels that have arrived. The **Retrain policy** retrains every 14 days, plus a safety net. The **Trainer** builds a new model (the challenger). The **Promotion gate** compares it with the live model (the champion). If it passes, **MLflow** registers it and it becomes the new champion. The **Dashboard** shows everything.
Safety net	Retrains early if live PR-AUC falls 30% below normal, or close to the level of random guessing.
Window selection	Each retrain trains two challengers, last 60 days and all history, and keeps the one that scores better on the newest unseen labels.
Promotion gate	Promotes only if the new model is at most 0.01 PR-AUC worse than the current one, measured again through the real production features.

> **Say it in one line:** A weekly loop: monitor, decide, retrain, check, then serve.

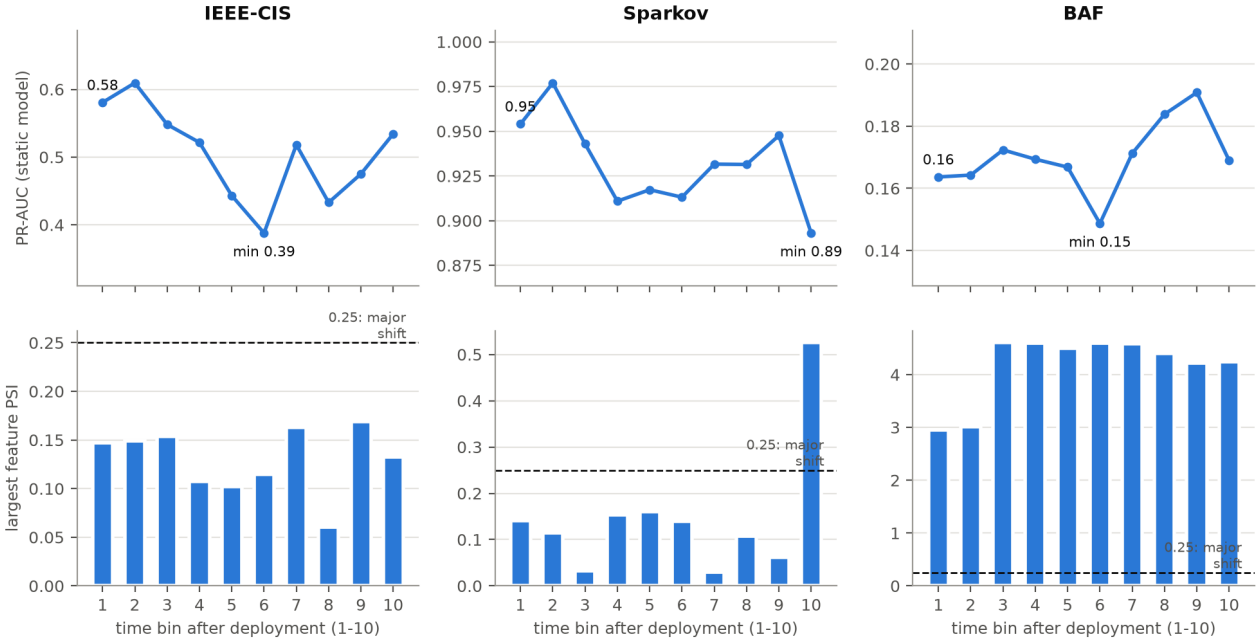
Part C (continued). The result figures

Before the graphs: what is PR-AUC? The model gives every transaction a fraud score. Sort them from highest to lowest. PR-AUC (between 0 and 1) says how well the real frauds sit at the top of that list. 1 is perfect. A model that guesses randomly gets about the fraud rate (0.035 on IEEE-CIS, 0.011 on BAF). That is why BAF's 0.16 is actually about 15 times better than guessing. We use PR-AUC instead of accuracy because with 99% normal transactions, a model that says 'never fraud' is 99% accurate but useless.

What is PSI? Population Stability Index: how much a feature's values have changed compared to the training period. Below 0.1 = stable, 0.1 to 0.25 = moderate, above 0.25 = major change (the usual alarm line).

Error bars (the thin vertical lines with caps): the 95% confidence interval. If the bar does not cross zero, the difference is reliable.

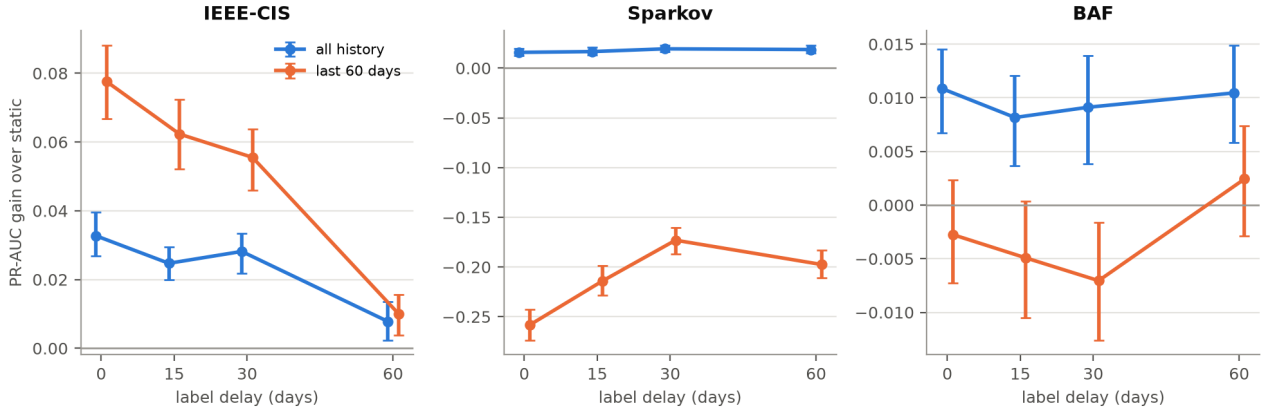
Result 1. Drift measures do not track performance



Question	When the model gets worse, does the drift alarm notice?
What it shows	One column per dataset. **Top row**: PR-AUC of a model trained once and never updated, over 10 time periods after deployment. **Bottom row**: the PSI of the most changed feature. The dashed line at 0.25 is the 'major change' alarm.
How to read	Compare top and bottom in the same column. If the alarm worked, bars would rise above the dashed line exactly when the line goes down.
Numbers	**IEEE-CIS**: PR-AUC drops from 0.58 to 0.39, but PSI stays below 0.17, never reaching 0.25: a **missed alarm**. **BAF**: PSI is 3 to 4.5, about 15 times the alarm line, but PR-AUC stays around 0.15-0.19: a **false alarm**. **Sparkov**: PR-AUC stays high (0.89-0.98); PSI crosses the line only in the last period.
Why this chart	Lines show change over time; bars show an amount at each time. Two rows because PR-AUC and PSI have different scales.

Say it in one line: Drift alarms missed real damage on IEEE-CIS and raised false alarms on BAF.

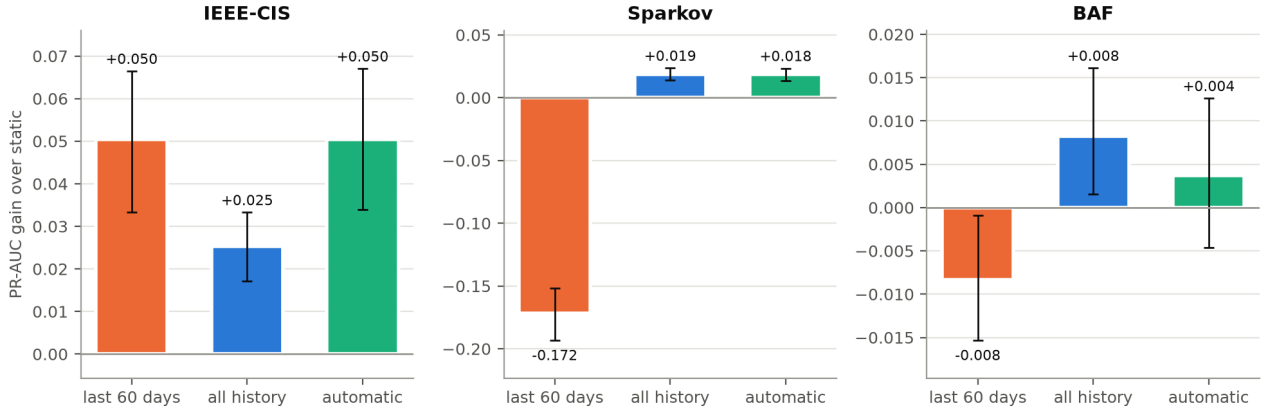
Result 2. Label delay erodes retraining



Question	If labels arrive late, how much does retraining still help?
What it shows	X axis: label delay (0, 15, 30, 60 days). Y axis: how much better retraining is than the never-updated model (PR-AUC gain). Blue = train on all history, orange = train on the last 60 days. Error bars = 95% confidence.
How to read	Above the zero line means retraining helps; below means it hurts. A falling line means the benefit shrinks as labels get later.
Numbers	**IEEE-CIS**, last 60 days: +0.078 at 0 days, falling to +0.010 at 60 days. All history: +0.033 to +0.008. **Sparkov**: last 60 days is far below zero (-0.17 to -0.26) because 60 days hold too few frauds; all history is about +0.02. **BAF**: all history about +0.01; last 60 days around zero or slightly negative.
Why this chart	A line shows the trend as delay grows; error bars show whether differences are real.

Say it in one line: The later the labels, the less retraining helps, and the right window depends on the dataset.

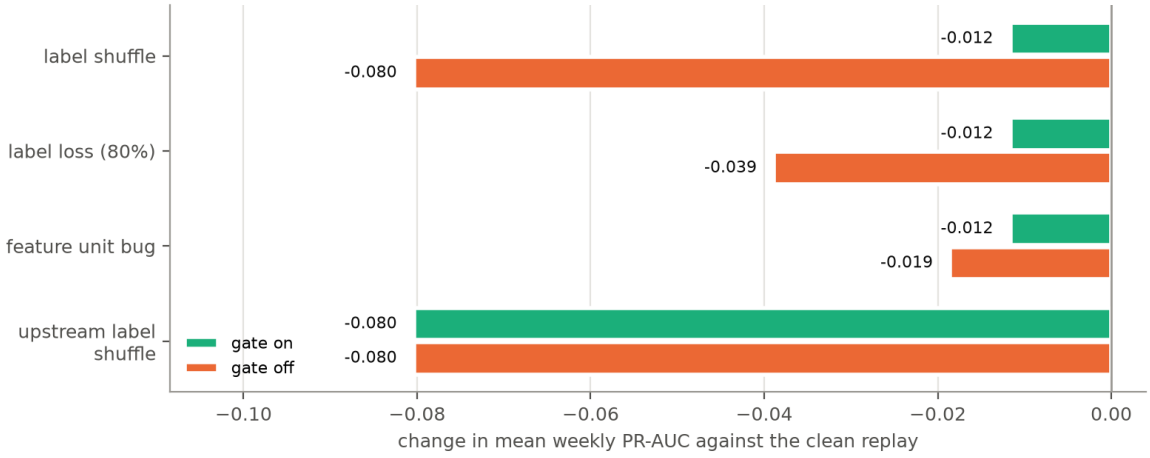
Result 3. Automatic window selection



Question	Can the framework pick the training window by itself?
What it shows	Full framework replays with a 30-day label delay. Three bars per dataset: always last 60 days (orange), always all history (blue), automatic choice (green). Height = PR-AUC gain over the never-updated model.
How to read	The best fixed window differs per dataset. Look whether green is close to the taller of orange and blue.
Numbers	**IEEE-CIS**: 60 days +0.050, all history +0.025, automatic +0.050. **Sparkov**: 60 days -0.172, all history +0.019, automatic +0.018. **BAF**: 60 days -0.008, all history +0.008, automatic +0.004 (its error bar crosses zero, so roughly equal to no change).
Why this chart	Bars are the clearest way to compare a few options side by side.

Say it in one line: Automatic selection matched the best fixed window without knowing it in advance, and avoided the big loss on Sparkov.

Result 4. The promotion gate



Question	Does the gate stop a bad model from going live?
What it shows	We broke the training data on purpose in four ways. For each, the loss in average weekly PR-AUC compared with a clean run. Green = gate on, orange = gate off. Longer bar = more damage.
The faults	**Label shuffle**: fraud labels mixed up randomly. **Label loss (80%)**: most labels missing. **Feature unit bug**: a feature in the wrong unit (like cents instead of dollars). **Upstream label shuffle**: labels corrupted at the source, so both models are judged on wrong labels.
Numbers	Gate off: -0.080, -0.039, -0.019. Gate on: only -0.012 each time. Upstream shuffle: -0.080 either way.
Why the last row is the same	The gate checks models with the labels it has. If the labels themselves are wrong at the source, the gate is fooled too. That case needs the safety net and data-quality checks. Be honest about this; it shows you understand the limit.

Say it in one line: The gate blocked every bad model trained on broken data, cutting the damage from up to 0.080 to 0.012.

9. Conclusion and Future Work

Poster point	Meaning in simple words
Drift indicators can miss real degradation under delayed labels	The usual alarms watch the inputs, not the results. They missed the real drop on IEEE-CIS and cried wolf on BAF.
Drift-aware = tracking drift through delayed-label performance	We still care about drift, but we judge it by how the model actually performs on the labels that have arrived.
Equal retraining budget	Supervisor's question: does the schedule win only because it retrains more? We gave both the same number of retrains. On IEEE-CIS (60-day window) the schedule was better in 18 of 27 pairs and the trigger was never reliably better; on BAF with a 30-day delay the schedule also led.
Window selection and promotion gate	Retraining adapts to each dataset and stays safe from broken data.
API load test (done)	Supervisor's request. The first run found that converting the input took almost all of the time (about 14 requests per second). After the fix, with identical scores: about 1,300 requests per second, about 1 ms for one transaction, p95 latency 82 ms with 100 clients at the same time, no errors.
Next	Equal-budget test on Sparkov; repeat the load test on a server with several worker processes.
Later	Faster investigator feedback, other model families, monitoring of explanations (SHAP), a live deployment.

Part D. Numbers to remember

What	Number
Datasets	3: IEEE-CIS (590,540 rows), Sparkov (1,801,969), BAF (1,000,000)
Fraud rates	3.5%, 0.53%, 1.1%
Label delays tested	0, 15, 30, 60 days
Retrain schedule	every 14 days; safety net at most once per 7 days
Training windows	last 60 days, or all history; chosen automatically
Safety net	30% drop below normal, or near random-guess level
Promotion gate	new model may be at most 0.01 PR-AUC worse
Decay on IEEE-CIS	PR-AUC 0.58 to 0.39, PSI never above 0.25
Retraining gain, IEEE-CIS	+0.078 (no delay) to +0.010 (60-day delay)
Gate effect	damage -0.080 without gate, -0.012 with gate
API load test	about 1,300 requests/s; 1 ms per transaction; p95 82 ms at 100 clients (before the fix: 14 requests/s)
Robustness	5 random seeds, walk-forward tuning, bootstrap confidence intervals
Tools	Python, LightGBM, pandas, scikit-learn, MLflow, FastAPI, Kaggle

Part E. Words in simple language

Word	Simple meaning
Concept drift	The data or the fraud patterns change over time, so an old model gets worse.
Label delay	We learn whether a transaction was fraud only weeks later.
MLOps	Running machine learning models in real use: deploy, watch, update, roll back.
Retraining	Training a fresh model on newer data.
Scheduled retraining	Retrain at fixed times (every 14 days) no matter what.
Drift-triggered retraining	Retrain only when an alarm says the data or performance changed.
Training window	How much past data the new model learns from.
Champion / challenger	The live model / the newly trained model that wants to replace it.
Promotion gate	A check that the challenger is not worse before it goes live.
MLflow	A tool that stores every model version and its scores.
FastAPI	A tool for building the web service that scores transactions.
Replay	Feeding old data week by week as if it were happening now.
Fault injection	Breaking the data on purpose to test safety.

Part F. Questions you may get (short answers)

Question	Answer
Why not use accuracy?	Fraud is about 1%, so 'never fraud' is 99% accurate but useless. PR-AUC focuses on catching frauds at the top of the alert list.
Why is BAF's PR-AUC so low (0.16)?	BAF fraud is very hard to separate and only 1.1%. Random guessing gets 0.011, so 0.16 is about 15 times better.
Did you use a pre-trained model?	No. LightGBM is trained from scratch on each dataset, hundreds of times.
Why LightGBM and not deep learning?	On table data boosted trees are usually the strongest, and they are fast enough to retrain many times. Other models are future work.
Why did you not shuffle the data?	Shuffling mixes the future into training and hides drift. Real banks always learn from the past to predict the future.
Why does the schedule beat the drift trigger?	The trigger needs proof that performance dropped, and that proof comes from labels that are weeks old. The schedule keeps the model fresh without waiting.
Is the schedule better only because it retrains more?	No. With the same number of retrains, the schedule still matched or beat the trigger under delayed labels.
Why 60 days on IEEE-CIS but all history on Sparkov?	IEEE-CIS changes fast, so old data misleads. Sparkov is stable and fraud is rare, so 60 days have too few frauds.
What happens if the gate is fooled?	If labels are corrupted at the source, both models look the same. The safety net and data checks handle that case.
What is new in your work?	A realistic test with label delay, a fair equal-budget comparison, and a framework with delayed-label monitoring, automatic window selection and a gate that re-scores on production features.
Is it real time?	We replay historical data week by week, but the scoring API is real: in the load test it answered one transaction in about 1 ms and handled about 1,300 requests per second. A live stream is the next step.
What did the load test find?	At first only 14 requests per second, because turning the JSON into a pandas table took 57 ms. We now convert the input straight into numbers: same scores, about 100 times faster.
What is p95 latency?	The time within which 95 of every 100 requests finish. It matters more than the average, because the slowest requests keep customers waiting.
Was there any data leakage? Did you check?	We kept strict time order and used only labels the bank would have, with unit tests. One known leak is in IEEE-CIS's own labelling: fraud is spread to the same client's later transactions. We rebuilt a client ID and checked: without those clients the retraining gain halves (0.056 to 0.027) but stays reliable, and the ranking of the strategies does not change.